



FEATURE MODEL VISUALIZATION: PROBLEMATIC ARCHER MODEL DIAGNOSTICS

Bachelor's Project Thesis

Reinder Kas, s3189953, r.kas@student.rug.nl,
Supervisor: Dr. H.A. de Weerd

Abstract: Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed at sodales magna, a tempus tellus. Sed ut dui imperdiet, pulvinar ligula nec, placerat orci. Nulla porttitor justo mi, vitae convallis nisl lacinia sit amet. Donec viverra pellentesque arcu, ut cursus mi commodo sit amet. Donec ut iaculis nibh, et feugiat enim. Ut sodales ipsum tellus, eu mattis sem.

1 Introduction

Within the field of Artificial Intelligence and Computer Science, there are many complex real-world problems that researchers try to solve. Many of these problems can be represented intuitively as Constraint Satisfaction Problems (e.g., scheduling or timetable problems). These problems consist of a set of Constraints given by the environment, or model, that the problem tries to emulate. These Constraints can be in many forms; from logical requirements to resource limitations (mostly used for optimization problems).

This paper will focus only on Constraint Satisfaction Problems that deal with the form of logical requirements called Feature Models.

1.1 Feature Model

Within the field of Software Development there is a compact way to represent all the products of the Software Product Line, called a Feature Model. This Feature Model is a visual way of modelling all the Relationships (Constraints) and Features (Variables) within a product, giving an overview of the structure dependencies and what is and is not possible within every aspect of a product.

Within this Feature Model, a Feature can be either selected (True) or not selected (False). Other values of the domain such as numeric values are also possible but are beyond the scope of this research paper. At the base of every Feature Model, there is a Root Feature. This is the Feature at the top of the Hierarchical tree, defining the starting point of

the Feature Model and the highest point of abstraction for the Model. We define the Root Feature to always be True, since the Root Feature being False would mean that this Feature is not present, thus making the entire Feature Model obsolete.

These Feature Models also have a predefined set of Constraints, typically divided into two relationship types (shown in table 1.1): the Parent-Child relationships, and the Cross-Tree relationships, sometimes also known as the Hierarchy relationships and the Cross-Hierarchy relationships (Benavides et al., 2010).

1.1.1 Parent-Child Relationship

The Parent-Child relationships are the relationships that define the structure and hierarchy within the model. Since these relationships are defined in this hierarchical structure, we know that if a Child Feature is selected, the Parent Feature must also be selected.

There are four main relationships that define this hierarchical structure as Parent-Child relationships:

Mandatory: The Parent and Child must have the same value (True or False).

Optional: The Child can only be selected iff the Parent is selected.

Or: There must be at least one Child selected iff the Parent is selected.

Alternative: There must be exactly one Child selected iff the Parent is selected.

1.1.2 Cross-Tree Relationship

The Cross-Tree relationships are relationships that are typically inclusion or exclusion statements. These relationships make it possible to define dependencies between Features throughout the whole hierarchy of the tree.

Requires: If Feature A is selected, then Feature B must be selected as well

Excludes: Both Features A and B cannot be selected at the same time.

The definition of the Feature Model Relationships between Parent P and Child C (A and B for Cross-Tree relationships) are given as follows: (Hotz et al., 2014a)

Constraint	Semantics
Mandatory	$P \leftrightarrow C$
Optional	$P \rightarrow C$
Or	$P \leftrightarrow (C_1 \vee C_2 \vee \dots \vee C_n)$
Alternative	$(C_1 \leftrightarrow (\neg C_2 \wedge \dots \wedge \neg C_n \wedge P))$ $\wedge (C_2 \leftrightarrow (\neg C_1 \wedge \dots \wedge \neg C_n \wedge P))$ $\wedge \dots$ $\wedge (C_n \leftrightarrow (\neg C_1 \wedge \dots \wedge \neg C_{n-1} \wedge P))$
Requires	$A \rightarrow B$
Excludes	$\neg A \vee \neg B$

Table 1.1: Feature Model relationships (Hotz et al., 2014a) showing the semantics of the Hierarchical Constraints (white background) and the Cross-Hierarchy Constraints (light-gray background).

At first glance, it might seem confusing that the Optional and Requires Constraints have the same implementation, but it is important to separate these definitions. Making a separation between this Parent-Child relationship and the Cross-Tree relationship makes it easier to show and define Hierarchy in a model. Without this distinction, the hierarchy of a model will be ill-defined.

One example of a Feature Model that is often used to showcase the usage of all of these Constraints is the Mobile Phone Feature Model as

shown below in Figure 1.1.

This model gives a compact and intuitive representation of a Mobile Phone such that the underlying relationships between all the Features within the Feature Model are visible in a visual representation.

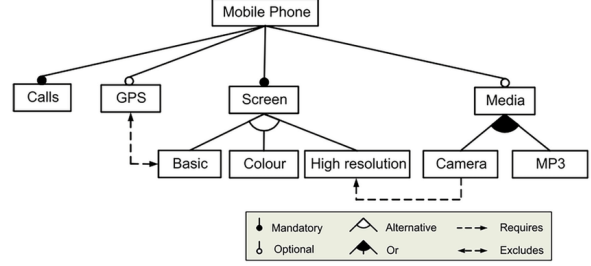


Figure 1.1: Feature Model representation of a simple phone (Benavides et al., 2010).

1.2 Constraint Satisfaction Problem

A Constraint Satisfaction Problem is a mathematical problem (model) that consists of three components X, D, and C: (Russell & Norvig (2010))

X is a set of variables, $\{X_1, \dots, X_n\}$.
D is a set of domains, $\{D_1, \dots, D_n\}$.
C is a set of constraints that specify allowable combinations of values.

If a model is created with these three components then a Solver will be able to find a solution to the problem as long as one exists. Many of these solvers exist already, such as Microsoft Z3 Solver (de Moura & Bjørner, 2008), Google OR-Tools (Perron, 2011), or choco-solver (Prud'homme et al., 2019). Each of these solvers works in a different way but should ultimately come to the same solution if the problem is defined correctly.

There can also be cases where a model does not have a solution:

$X = \{A\};$
 $D = \{A = \{True, False\}\};$
 $C = \{(A \neq A)\};$

The solution to this defined model should satisfy the Constraint $A \neq A$. Since we know that a Variable can have only a single value assigned to it at

a given moment, it is immediately clear that there will be no solution to this model. We cannot assign Variable A such that it is not equal to its own value. This means that the defined model is unsatisfiable.

In this small example, it is clear where the fault in the model lies. There is only a single Constraint that we impose on the problem. To make the problem satisfiable again we can either change or remove this Constraint. If the model were to be bigger, finding out where the problem is within the logic of the model would become more difficult. More Constraints means more complexity and more possibilities of conflict within the model's logic.

For this complexity of finding the problem within a Constraint model it would be ideal to have an interpreter or visualizer such that it is immediately clear where the problem within a model's logic lies.

2 Problem Description

2.1 Elfsquad

Elfsquad is a company that specializes in online Product Configurations. The basic principles that satisfy most requirements for their customers consist of Constraints that are only logical Constraints. After having developed their own software that allows customers to create their own Feature Models, it became apparent that these models become complex very quickly.

The typical Feature Model within the Elfsquad software consists of 1.000 Features and 1.500 Constraints (upwards to 12.000 Features and 25.000 Constraints). Their customers often have problems with their Feature Models, where these models do not have a solution, meaning their model is unsatisfiable thus they cannot use Elfsquad's software to sell their product. This is a massive problem for customers because not being able to configure a product means a loss of revenue with the wasted time that could have been spent selling a product. In a lot of these cases, the customers call Elfsquad support to troubleshoot their problems, which in turn now also costs Elfsquad extra resources that could and should have been avoided in the first place.

As of right now, Elfsquad's support spends too much time diagnosing Feature Models that do not have a solution. The main problem here is that

these models that do not have a solution should not have been created in the first place. If they are created by customers, then there should exist a tool that empowers the customer to self-diagnose the problem within their own Feature Model. If this tool is easy enough to use and shows the correct amount of information, then the amount of time for the Elfsquad support will spend diagnosing Feature Models should greatly reduce, giving them more time to help with different matters.

Visualization tools already exist for Feature Models, such as FeatureIDE (Thüm et al., 2014), but these visualization tools do not give enough functionality for the Elfsquad company to integrate with. Elfsquad has created its own method of creating Feature Models through its in-house developed Archer Modeling Language. This Archer Modeling Language is more expansive than the well-known Feature Model Constraints that FeatureIDE relies on to function. This means that a different solution must be found.

2.2 Archer

Elfsquad has developed its own Modeling Language, known as Archer Modeling Language. This language takes a String-based input and generates a Microsoft Z3 Solver object containing the problem definitions as per the input string.

Since this language is a String-based language, there is no visual representation of what the logic within a Feature Model means. Even smaller Feature Models could become problematic to diagnose since all information pertaining to the model is written in a textual format.

Because other visualization tools do not have the functionality to support the Archer Modelling Language, it has been decided to create an in-house developed tool to give a visual way to represent these models. This thesis functions as the basis of this tool upon which will be further researched and developed in the future.

3 Method

3.1 Program

The visualization tool consists of two separate applications: the front-end and the back-end. These

are built using a Client-Server pattern and during the development process will run locally in Docker. The API is the Server that provides service and the interface is the Client requesting services. This pattern is chosen because it fits well with the existing implementations and philosophies of Elfsquad (API-first design).

Interface

The interface is created with AngularJS. This is a TypeScript-based, free and open-source single-page web application framework. This framework is also used by Elfsquad and allows for making components that can later be directly implemented into Elfsquad's source code without extra work. The visualization within the tool makes use of the D3 JavaScript library (Rothenhäusler, 2022).

API

The API is created in .NET 7.0. As of writing this thesis, this is the newest version of .NET that exists. Since the Archer Modeling Language makes use of the Microsoft Z3 Solver it is best to use the Z3 solver for this diagnostics tool as well.

Upon initializing a new Z3 solver object, it is possible to set flags that determine the behaviour of the Z3 Solver. The two flags most relevant for what this tool wants to achieve are the flags **proof**, and **unsat_core**.

The **proof** flag tells the Z3 Solver to generate a proof in Lisp showing the internal logical structure of the Z3 Solver as to why it could not satisfy the given Constraints.

The **unsat_core** flag tells the Z3 Solver to generate the minimal set of Constraints that is still unsatisfiable (Unsatisfiable Core).

This tool will have multiple functionalities. It will show the proof generated by the Z3 solver in Lisp and also will produce a visual representation of the entire given Feature Model and highlight the Constraints that are present within the Unsatisfiable Core. This shows the end-user where the problem in the model is according to the Z3 Solver in a graphical way. It will also generate a list of Constraints that have to be removed to make the problem satisfiable again according to the Fast Diagnose algorithm (?).

3.2 Experiment

(TODO: MAYBE CROSS TEST WITH ARCHER MODEL / Z3 PROOF / GRAPH / FAST DIAGNOSE?)

In order to test the effectiveness of this tool, 10 Elfsquad employees will have to diagnose 10 predefined problematic Archer Models.

The models are increasing in size, starting with a small model and increasing to the biggest model.

Each employee will have to diagnose the problem within all 10 models.

There are two methods of diagnosing this problem. One method would be to use the Archer Modeling Language model definition, and the other method will be to use the graph visualization tool that is created for the Archer Modeling Language. Each employee's method of diagnosing the model will be randomized such that they have 5 Archer Language models, and 5 graphical visualized models. All models will also have 5 employees diagnosing the model as Archer representation and 5 as graph representation to make sure all model results can be compared to each other correctly.

This method allows for comparison between the time it takes the Elfsquad employees to diagnose the problem within each model in the Archer representation and the graphical representation.

4 Design Choices

4.1 Proof

The initial research was done into making use of the Proof generated by the Microsoft Z3 solver in order to decipher why a Feature Model does not have a solution. After implementing the Z3 solver and generating this proof, it quickly became clear that it is not a feasible method of finding the solution to a problematic Feature Model for this thesis. A small example Feature Model (hereafter referenced as Simple Model) is defined as:

$$\begin{aligned} \text{Root} &\xrightarrow{\text{Mandatory}} \text{Child} \\ \text{Child} &\xrightarrow{\text{Excludes}} \text{Child} \end{aligned}$$

Or in the CSP notation:

$$\begin{aligned}
X &= \{Root, Child\}; \\
D &= \{Root = \{True\}, \\
&\quad Child = \{True, False\}\}; \\
C &= \{(Root == Child), \\
&\quad (\neg Child \vee \neg Child)\};
\end{aligned}$$

Here we have the situation where the Root Feature must be selected. The Mandatory Constraint implies that the Child must also be selected. The Exclude Constraint implies that if the Child Feature is selected, then the Child Feature cannot be selected. This means that there is no way to assign the Child Feature a value such that it will satisfy all Constraints present in the problem set.

The intuitive method of resolving this problem is to remove the Exclude Constraint. The proof generated by the Z3 solver as seen in Figure 4.1 does not show this intuitively at all and is significantly big already.

```

(let ((a!1 (def-axiom (or (not (= Root Child)) (not Root) Child)))
      (a!2 (monotonicity (rewrite (= (= Root true) Root))
                        (= (=> proxy!2 (= Root true)) (= proxy!2 Root))))
      (a!3 (rewrite (= (=> proxy!2 Root) (or Root (not proxy!2))))))
      (a!6 (asserted (=> proxy!1 (or (not Child) (not Child))))))
      (a!7 (rewrite (= (or (not Child) (not Child)) (not Child))))
      (a!8 (= (=> proxy!1 (or (not Child) (not Child)))
              (= proxy!1 (not Child))))
      (a!9 (rewrite (= (=> proxy!1 (not Child)) (or (not Child) (not proxy!1))))))
      (a!10 (= (=> proxy!1 (or (not Child) (not Child))
                       (or (not Child) (not proxy!1))))))
      (a!12 (rewrite (= (=> proxy!0 (= Root Child))
                       (or (not proxy!0) (= Root Child))))))
      (let ((a!4 (trans a!2
                      a!3
                      (= (=> proxy!2 (= Root true)) (or Root (not proxy!2))))))
            (a!11 (unit-resolution (mp a!6
                                       (trans (monotonicity a!7 a!8) a!9 a!10)
                                       (or (not Child) (not proxy!1)))
                                   (asserted proxy!1)
                                   (not Child))))
            (a!13 (mp (asserted (=> proxy!0 (= Root Child)))
                      a!12
                      (or (not proxy!0) (= Root Child))))))
      (let ((a!5 (mp (asserted (=> proxy!2 (= Root true)))
                    a!4
                    (or Root (not proxy!2))))))
            (unit-resolution a!1
                            (unit-resolution a!5 (asserted proxy!2) Root)
                            a!11
                            (unit-resolution a!13 (asserted proxy!0) (= Root Child))
                            false))))

```

Figure 4.1: Proof generated by the Microsoft Z3 solver for the Simple Model.

The size and complexity of this proof only increase as the complexity of the Feature Model increases as well (see appendix TODO: ADD MODELS AND PROOFS INTO APPENDIX). Therefore the decision has been made to research other options for diagnosing the Archer Models first.

4.2 Unsatisfiable Core

The next point that was researched was how to make use of the so-called Unsatisfiable Core. This Unsatisfiable Core is the minimal set of Constraints in the Feature Model that produce an unsatisfiable result. This minimal set is thus guaranteed to contain the problem of the Feature Model and can be seen as the main focus point for diagnosing the problem within the unsatisfiable model.

Constraints in Unsatisfiable Core:

(= Root Child)

(or (not Child) (not Child))

(= Root true)

Figure 4.2: Unsatisfiable Core generated by the Microsoft Z3 solver for the Simple Model.

This generated Unsatisfiable Core is already a way more uncluttered way of indicating where the problem in the model lies. The problem here is that this result is still written in Lisp, which is a language that the end users are most likely not familiar with. This can cause confusion for the user which means that the tool should be even more intuitive.

4.3 Graph

The step following generating the Unsatisfiable Core is to show this result in a visual representative method. This can be done in a variety of ways, but for this thesis, it has been chosen to make use of the D3 JavaScript library. This library makes it easy to draw a graph on a canvas which shows in the blink of an eye where the problem in the model lies.

The entire Feature Model is taken and drawn into a graph. Then the Unsatisfiable Core that is generated by the Z3 solver is taken. All the Features that are present within this Unsatisfiable Core are highlighted as red Nodes in the Graph to indicate that there is something wrong with them. All other nodes are highlighted as green to indicate that these Nodes do not contribute to the Feature Model being unsatisfiable.

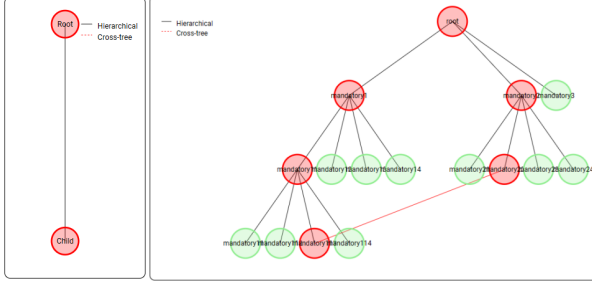


Figure 4.3: Graphical overview of the Unsatisfiable Core overlapped onto the Feature Model representation with the Simple Model (left) and a slightly more complex model (right).

In this view, it becomes immediately clear where the problem in the Feature Model lies in more complex models. The Feature Model on the left is the aforementioned Simple Model where both the Root and the Child Feature are present and the cause of the unsatisfiability, thus the entire model is highlighted in red.

In the model shown on the right, there are more Features and Constraints that are present to show that not the entire Feature Model has to be the cause of the unsatisfiability. The red line between the red Features indicates an Exclude relationship and all other lines are Mandatory relationships. From this graph, it is clear where the Exclude is that causes the Feature Model to be unsatisfiable.

4.4 Interactive Graph

Another method of visualizing the graph to the end user that has been researched is an interactive graph. This interactive graph allows the user to drag and drop Features such that the user can create its own structure. This interactive graph has been created, but after using it, it has been decided that the additional information that this graph pro-

vides is of such low value for now that it has been decided to halt this research and continue investing time into the Unsatisfiable Core and Fast Diagnose paths of this research.

4.5 Fast Diagnose

In order to give the user of the tool an example of how to make the defined Feature Model satisfiable again, the Fast Diagnose algorithm (Hotz et al., 2014b) is implemented as seen in 4.1. This Fast Diagnose algorithm is a divide and conquer algorithm. It takes both the Cross-Tree Constraints (C) and all the Constraints present in the Feature Model (AC) and tries to find the Cross-Tree Constraints that cause the Feature Model to be unsatisfiable. By dividing these Cross-Tree Constraints (C_1, C_2) and systematically removing these from the Constraints that are checked to be satisfiable, we end up with the Constraints that are the cause of trouble within the Feature Model.

For example; if AC is unsatisfiable, but $AC - C_1$ is satisfiable, then we know that the problem in the model lies within the C_2 set of Constraints. Since this set is unsatisfiable, through recursion this same logic is applied again. In the next iteration of the recursion, the variable AC is now taken as $AC - C_2$ (which is the same as $AC \cup C_1$), C becomes C_2 and now divide this newly given C up into two new sets (C_1, C_2). This recursion is applied until one or more problematic Constraints are found and we return these Constraints as a solution.

Algorithm 4.1 The Fast Diagnose algorithm (Hotz et al., 2014b)

```

1: function FASTDIAGNOSE( $C \subseteq AC, AC = \{c_1..c_t\}$ )
2:   if (ISEMPTY( $C$ ) || !HASOLUTION( $AC - C$ )) then
3:     return  $\emptyset$ 
4:   else
5:     return FD( $\emptyset, C, AC$ )

6: function FD( $D, C = \{c_1..c_q\}, AC$ )
7:   if ( $D \neq \emptyset$  && HASOLUTION( $AC$ )) then
8:     return  $\emptyset$ 
9:   if SINGLETON( $C$ ) then
10:    return  $C$ 
11:    $k \leftarrow \frac{C}{2}$ 
12:    $C_1 \leftarrow \{c_1..c_k\}$ 
13:    $C_2 \leftarrow \{c_{k+1}..c_q\}$ 
14:    $D_1 \leftarrow$  FD( $C_2, C_1, AC - C_2$ )
15:    $D_2 \leftarrow$  FD( $D_1, C_2, AC - D_1$ )
16:   return  $D_1 \cup D_2$ 

```

Once the minimal conflict set is obtained from this algorithm, the tool can present the resulting Constraints to the end user in order to inform them which Constraints are the exact Constraints that cause problems with the Feature Model. The user then has the option to either change or remove these Constraints.

(TODO: MAKE LISP RESULT MAYBE? IT'S IN ARCHER MODELING LANGUAGE NOW)



Figure 4.4: The result of the Fast Diagnose on the Simple Model that is presented to the end user of the tool.

5 Results

(TODO: GRAPH/TABLE OF RESULTS)

6 Conclusion

This thesis is trying to find a method to reduce the time spent on diagnosing problems within a Feature Model that does not have a solution. A visualization tool has been created to reduce the time that it will take to find the problematic logic within a Feature Model.

This tool presents the user with a variety of methods to know where the problem lies. The most difficult one is to use the proof that is generated by the Microsoft Z3 solver. It is deemed easier to just diagnose through the input String and see where the problem is. The graph is a very easy tool to see which Constraints are involved within the problem set, but the result of the Fast Diagnose algorithm is by far the most effective. This result immediately shows which Constraint to delete in order to make the Feature Model satisfiable again.

7 Critique

With this thesis the visualization tool has only been tested and used by employees of Elfsquad. The ultimate goal is to have the customers of Elfsquad to use the tool instead of the employees. This is not possible yet, as the Archer Modeling Language itself is still in development and set to be released later on. The customers are scheduled to receive access to this tool at the same time as access to the Archer Modeling Language.

Furthermore, as of right now the newest version of the Archer Modeling Language has not been implemented into the tool yet.

The Archer Language is an ongoing project developed in-house at Elfsquad. Keeping up to date with the newest version of Archer would be too much implementation jumping from version to version that the decision has been made to take an older version of Archer and implement this into the tool.

Once Archer is completed, the tool is operational, and the tool has shown the company that the tool is able to increase the operational workflow for Elfsquad, then the complete version of Archer can be implemented into the tool for future use.

Another point of critique is that this tool is only tested for relatively small Feature Models (TODO: ADD FEATURES AND CONSTRAINT COUNT HERE.). The actual Feature Models used in Elfsquad are many times larger than the ones used in this thesis. There is no way of knowing if these larger models will still be visually easy to debug if the scale becomes that much larger, but the assumption is that it will be. The only way to really know this is to implement this tool into the Elfsquad software and actually use it in the Feature Models it is intended for.

8 Future Research

This thesis functions as the basis of a diagnostics tool for the Elfsquad Archer Modeling Language. This means that future research depends on the implementation of the Archer Modeling Language and the ways that Elfsquad's customers want to use the tool. Since Archer is still in development within Elfsquad, there is no saying

exactly what the customers will come up with and how they want to use it. This however does not mean that no future research can be thought of:

Interpret the Z3 generated proof.

As of right now, nothing is being done with the Z3-generated proof except for being shown to the end user. This proof does contain all the information as to why it is not possible to find a solution and perhaps an easier interpretation can be created for diagnosing problematic Feature Models using this proof.

Complete Archer Modeling Language implementation.

For now, there is only a limited component of the Archer Modeling Language implemented in the tool. Ongoing development into Archer has prevented this tool from keeping up with the latest version, as it would be too much work to keep up with all and every change as they come.

Generate solutions to the problem.

The tool shows the end user how to fix the problem in the Feature Model by removing one or multiple Constraints provided by the Fast Diagnose algorithm. Removing the Constraint is usually not the desired way to fix the Feature Model. It would be more useful to generate different solutions to the problem as well (e.g., making a Mandatory relationship Optional). This generation of solutions can be done in multiple of ways but is planned for further development of this tool.

Implement Numeric Domains.

This tool currently only works with Features that make use of Boolean Domains. A Feature can be selected or not. Elfsquad has encountered Feature Models where customers want to make use of Numeric Domains as well. These Numeric Domains are acted upon by different Constraints, ones that are not present in the simple Boolean Feature Models that are discussed in this paper.

In order to support these Constraints, new visualizations should be added to the tool in order to give

information pertaining to the numeric Constraints in the Model.

Make the interactive graph more useful.

The interactive graph does not provide enough additional functionality as of now to be deemed useful. This graph can be built upon such that it does provide this. In the future more research can be done with bigger and more complex models where a structured overview is necessary in order to make sense of the problem within the Feature Model. As of now the Feature Models that have been tested are not complex enough to make this interactive graph feasible.

References

- Benavides, D., Segura, S., & Ruiz-Cortés, A. (2010, 09). Automated analysis of feature models 20 years later: A literature review. *Information Systems*, 35, 615-636. doi: 10.1016/j.is.2010.01.001
- de Moura, L., & Bjørner, N. (2008, 04). Z3: an efficient smt solver. In (Vol. 4963, p. 337-340). doi: 10.1007/978-3-540-78800-3_24
- Hotz, L., Felfernig, A., Stumptner, M., Ryabokon, A., O'Neill, C., & Wolter, K. (2014a, 04). Chapter 6. configuration knowledge representation and reasoning. In (p. 41-72). doi: 10.1016/B978-0-12-415817-7.00006-2
- Hotz, L., Felfernig, A., Stumptner, M., Ryabokon, A., O'Neill, C., & Wolter, K. (2014b, 04). Chapter 7. configuration knowledge representation and reasoning. In (p. 41-72). doi: 10.1016/B978-0-12-415817-7.00006-2
- Perron, L. (2011, 01). Operations research and constraint programming at google. In (p. 2). doi: 10.1007/978-3-642-23786-7_2
- Prud'homme, C., Fages, J.-G., & Lorca, X. (2019). Choco solver. *Website, March*.
- Rothenhäusler, L. (2022). *d3.js and its potential in data visualization* [bachelorthesis]. doi: 10.25933/opus4-2853
- Russell, S. J., & Norvig, P. (2010). *Artificial intelligence: A modern approach*. Prentice Hall.
- Thüm, T., Kästner, C., Benduhn, F., Meinicke, J., Saake, G., & Leich, T. (2014). Featureide: An extensible framework for feature-oriented software development. *Science of Computer Programming*, 79, 70-85. (Experimental Software and Toolkits (EST 4): A special issue of the Workshop on Academic Software Development Tools and Techniques (WASDeTT-3 2010)) doi: <https://doi.org/10.1016/j.scico.2012.06.002>